

BOOK 05 — JAVA COLLECTIONS FRAMEWORK

Beginner to Senior Interview Mastery (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 05 of 24 **Java Versions**

Covered: Java 7/8 baseline (HashMap treeification in Java 8), Java 9 (immutable `List.of()` / `Set.of()` / `Map.of()`), Java 11/21 usage notes **Prerequisites:** Book 01 (arrays, generics-adjacent basics), Book 02 (equals/hashCode contract, interfaces), Book 03 (heap/object memory model) **Next Book:** 06_Java_Generics.md

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: Book 01 లో arrays fixed-size అని నేర్చుకున్నారు. ఈ పుస్తకం లో **dynamic, resizable data structures** — Collections Framework — పూర్తిగా నేర్చుకుంటాము: List, Set, Map, Queue, వాటి internal hashing/tree mechanics సహా. ఇది daily Java development లో అత్యంత ఎక్కువగా వాడే API, మరియు interview లో అత్యంత లోతుగా cross-question చేయబడే topic.

English: Book 01 gave you fixed-size arrays. This book covers the full Collections Framework — List, Set, Map, Queue — including their internal hashing/tree mechanics. This is the most-used API in daily Java development and one of the most deeply cross-questioned interview topics at every level.

Learning Objectives

1. Navigate the full Collections Framework interface hierarchy.
2. Master `ArrayList` / `LinkedList` internals and know when to use each.
3. Master `HashSet` / `LinkedHashSet` / `TreeSet` and their ordering guarantees.
4. Deeply understand `HashMap` internals: hashing, buckets, collisions, resizing, treeification.
5. Understand `LinkedHashMap` , `TreeMap` , `Hashtable` , and `ConcurrentHashMap` at a structural level.
6. Use `PriorityQueue` / `ArrayDeque` idiomatically.
7. Master `Comparable` vs `Comparator` .
8. Understand fail-fast vs fail-safe iterators.
9. Use the `Collections` utility class and immutable collection factories.
10. Choose the right collection for a given problem, backed by Big-O reasoning.

Table of Contents

Ch.	Title
1	Collections Framework — The Complete Hierarchy
2	ArrayList Deep Dive

Ch.	Title
3	LinkedList, Vector & Stack
4	HashSet Deep Dive
5	LinkedHashSet & TreeSet
6	HashMap Internals — Deep Dive
7	LinkedHashMap, TreeMap & Hashtable
8	ConcurrentHashMap — Structural Overview
9	Queue & Deque — PriorityQueue, ArrayDeque
10	Comparable vs Comparator
11	Iterators — Fail-Fast vs Fail-Safe
12	Collections Utility Class & Immutable Collections
13	Choosing the Right Collection — Big-O Decision Framework
14	Mini Project — In-Memory Inventory System
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — Collections Framework: The Complete Hierarchy

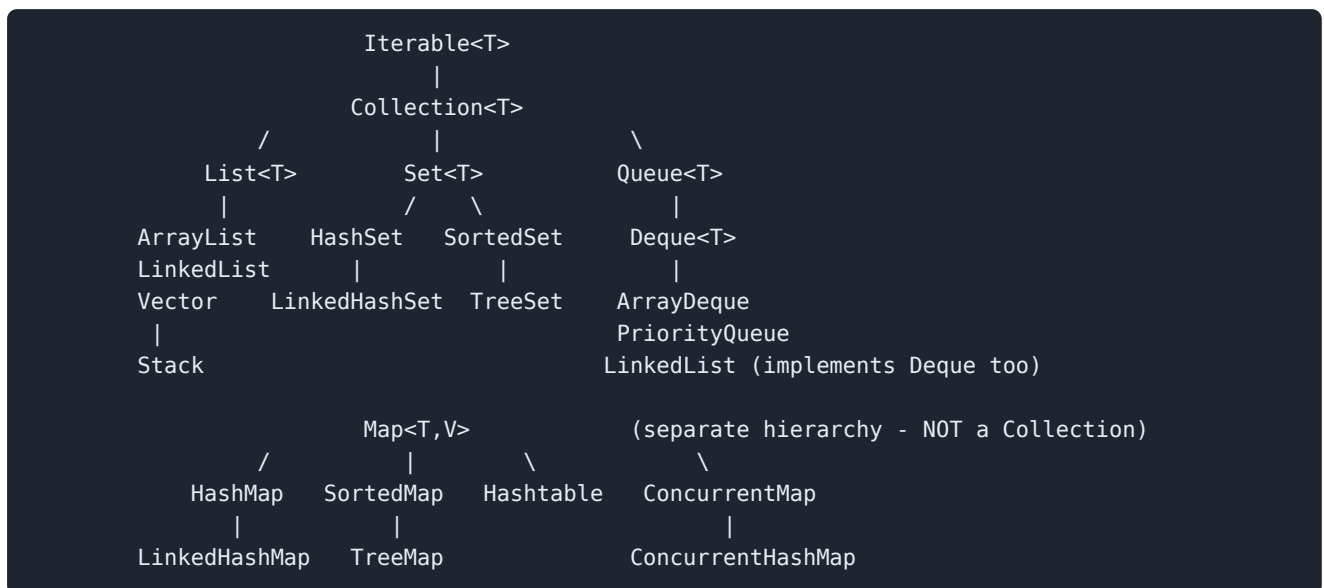
Telugu Explanation

Collections Framework అంటే Java లో group of objects ని handle చేయడానికి unified architecture — interfaces (`Collection` , `List` , `Set` , `Queue` , `Map`), వాటి implementations (`ArrayList` , `HashSet` , `HashMap` ...), మరియు algorithms (`Collections.sort()` , `Collections.binarySearch()`) కలిపి. గమనించండి: `Map` `Collection` interface ని extend చేయదు — ఇది key-value pairs కోసం వేరే hierarchy.

Professional English Explanation

The Collections Framework is Java's unified architecture for storing and manipulating groups of objects — a set of interfaces (`Collection` , `List` , `Set` , `Queue` , `Map`), their concrete implementations, and generic algorithms (`Collections.sort()` , `Collections.binarySearch()` , etc.). Note: `Map` is **not** a subtype of `Collection` — it models key-value associations and sits in its own parallel hierarchy.

Diagram — Full Hierarchy



Java Code

```
import java.util.*;

public class CollectionsOverviewDemo {
    public static void main(String[] args) {
        Collection<String> list = new ArrayList<>(List.of("a", "b", "c"));
        Collection<String> set = new HashSet<>(List.of("x", "y", "z"));
        Queue<String> queue = new LinkedList<>(List.of("p", "q"));
        Map<String, Integer> map = new HashMap<>(Map.of("one", 1, "two", 2)); // NOT a
        Collection<T>

        System.out.println("List: " + list);
        System.out.println("Set: " + set);
        System.out.println("Queue: " + queue);
        System.out.println("Map: " + map);

        System.out.println("Is Map a Collection? " + (map instanceof Collection)); // false
    }
}
```

Output (illustrative — Set/Map ordering isn't guaranteed here)

```
List: [a, b, c]
Set: [x, y, z]
Queue: [p, q]
Map: {one=1, two=2}
Is Map a Collection? false
```

Internal Working

- Every `Collection` implementation shares common operations (`add`, `remove`, `size`, `iterator`) declared on the `Collection` interface, inherited from `Iterable` (which is why every collection can be used in a for-each loop).
- `Map` was deliberately excluded from `Collection` in the original framework design because its core operation — associating a key with a value — doesn't fit the single-element-at-a-time contract (`add(E e)`) that `Collection` defines; instead it exposes `keySet()`, `values()`, and `entrySet()`, each of which **does** return a `Set` / `Collection` view.

Real-World Example

Telugu: REST API response ను `List<Product>` return చేయడం, cache ను `Map<String, Product>` వాడటం, unique tags కోసం `Set<String>` వాడటం — production backend code లో ఈ మూడు structures అత్యంత frequently కనిపిస్తాయి. English: Returning `List<Product>` from a REST endpoint, caching by ID with `Map<String, Product>`, and deduplicating tags with `Set<String>` are the three most common collection usages you'll write in real backend code — this chapter's hierarchy is the map you'll navigate for the rest of your career.

Interview Answer

"The Collections Framework provides `Collection` (parent of `List`, `Set`, `Queue`) and a separate `Map` hierarchy for key-value pairs — `Map` intentionally isn't a `Collection` because its add-a-pair semantics don't match `Collection`'s single-element contract, though it exposes `keySet()` / `values()` / `entrySet()` as collection views."

Cross Questions

- Q: Why isn't `Map` a subtype of `Collection`? → A: Its core `put(key, value)` operation doesn't match `Collection.add(E)`'s single-element contract; it's a fundamentally different abstraction (associations, not a flat group).
- Q: What's the difference between `List`, `Set`, and `Queue` at the interface level? → A: `List` allows duplicates and maintains insertion order with indexed access; `Set` disallows duplicates; `Queue` is designed for FIFO/priority-based processing order, not indexed access.
- Q: Does every `Collection` guarantee iteration order? → A: No — it depends on the specific implementation (`ArrayList` / `LinkedHashSet` / `LinkedHashMap` do; `HashSet` / `HashMap` don't guarantee any particular order).

Coding Exercise

L1: Create one `List`, one `Set`, one `Map`, and one `Queue`, printing each. **L2:** Use `map.keySet()`, `map.values()`, and `map.entrySet()` to iterate a `Map` three different ways. **L3:** Verify `Map` is not a `Collection` using `instanceof`, and explain what interface it belongs to instead. **L4 (Interview):** Draw the full hierarchy diagram from memory. **L5 (Senior):** Given a requirement (unique, sorted list of usernames with fast lookup), choose the right collection type(s) and justify your choice using this chapter's vocabulary alone (details come in later chapters). **L6 (Mastery):** Explain, without notes, why `Map` was deliberately excluded from the `Collection` hierarchy.

CHAPTER 2 — ArrayList Deep Dive

Telugu Explanation

`ArrayList` అనేది resizable array-backed `List` implementation. Internally ఒక `Object[]` array వాడుతుంది. Array నిండిపోతే, కొత్త, పెద్ద array (సాధారణంగా 1.5x size) create చేసి, old elements copy చేస్తుంది — దీన్నే **resizing/growth** అంటారు.

Professional English Explanation

`ArrayList` is a resizable-array-backed `List` implementation. Internally it holds an `Object[]` array. When capacity is exceeded, it allocates a new, larger array (typically 1.5x the old capacity in HotSpot's implementation) and copies existing elements over — this growth operation is amortized $O(1)$ per `add()` on average, though any single triggering `add()` is $O(n)$.

Java Code

```
import java.util.*;

public class ArrayListDemo {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();           // default initial capacity = 10
        for (int i = 0; i < 15; i++) list.add("item" + i); // triggers at least one resize

        System.out.println("Size: " + list.size());
        System.out.println("Get index 5: " + list.get(5)); // O(1) - direct array index

        list.add(3, "inserted");                          // O(n) - must shift elements
        System.out.println("After insert at 3: " + list.get(3));

        list.remove("item0");                              // O(n) - must find + shift
        System.out.println("Size after removal: " + list.size());

        List<String> preSized = new ArrayList<>(100);      // pre-sized to avoid repeated
        System.out.println("Pre-sized list is empty but capacity is reserved: " +
        preSized.isEmpty());
    }
}
```

Output

```
Size: 15
Get index 5: item5
After insert at 3: inserted
Size after removal: 15
Pre-sized list is empty but capacity is reserved: true
```

Internal Working — Growth & Complexity

Operation	Complexity	Why
<code>get(index)</code>	$O(1)$	Direct array index computation
<code>add(element)</code> (at end)	Amortized $O(1)$	Occasional $O(n)$ resize, averaged over many calls
<code>add(index, element)</code>	$O(n)$	All elements after <code>index</code> must shift right
<code>remove(index)</code> / <code>remove(Object)</code>	$O(n)$	Elements after the removed index shift left (plus $O(n)$ search for <code>remove(Object)</code>)
<code>contains(Object)</code>	$O(n)$	Linear scan, no hashing involved

- Default constructor starts with capacity 10 (some JDK versions lazily allocate on first `add()`); growth formula is roughly `newCapacity = oldCapacity + (oldCapacity >> 1)` (i.e., 1.5x).
- Pre-sizing via `new ArrayList<>(expectedSize)` avoids repeated array copies when the final size is known upfront — a genuine, measurable production optimization for large lists built in a loop.

Real-World Example

Telugu: REST API నుండి 10,000 records fetch చేసి List లో పెట్టేటప్పుడు, `new ArrayList<>(10000)` వాడితే multiple resize operations avoid అవుతాయి — performance-critical batch processing లో ఇది common optimization. English: When you know you'll load 10,000 records from an API/DB into a list, pre-sizing (`new ArrayList<>(10000)`) avoids repeated internal array copies — a small, well-known, measurable optimization in batch-processing code.

Interview Answer

"`ArrayList` is backed by a resizable `Object[]` array, offering $O(1)$ indexed access and amortized $O(1)$ append, but $O(n)$ insertion/removal at arbitrary positions since elements must shift. It grows by roughly 1.5x when capacity is exceeded, which is why pre-sizing when the final size is known avoids unnecessary array copies."

Cross Questions

- Q: Why is `get(index)` $O(1)$ but `remove(index)` $O(n)$? → A: `get` computes a direct memory offset; `remove` must physically shift every subsequent element one position left to keep the array contiguous.
- Q: Is `ArrayList` thread-safe? → A: No — concurrent modification requires external synchronization or a concurrent alternative (`CopyOnWriteArrayList`, Book 08).
- Q: What happens internally when `ArrayList.add()` triggers a resize? → A: A new, larger backing array is allocated, all existing elements are copied via `Arrays.copyOf` (or `System.arraycopy`), and the old array becomes garbage.

Tricky Questions

- Q: Does `new ArrayList<>()` immediately allocate an array of capacity 10? → A: In modern HotSpot implementations, no — it uses a shared empty array internally and only allocates the

real backing array (size 10 by default) on the first `add()` call, saving memory for lists that end up empty.

- Q: Is removing from the middle of an `ArrayList` in a loop using an index-based `for` loop safe?
→ A: It's error-prone — removing shifts subsequent elements down, so a naive incrementing index skips the next element; use an `Iterator.remove()` or iterate backward instead.

Coding Exercise

L1: Build an `ArrayList`, add 20 elements, and measure/compare `get(0)` vs `get(19)` conceptually (both $O(1)$). **L2:** Demonstrate the "removing while iterating with index" bug and fix it with `Iterator.remove()`. **L3:** Benchmark (via `System.nanoTime()`) inserting 10,000 elements with and without pre-sizing. **L4 (Interview):** Explain the growth formula and why amortized analysis makes `add()` " $O(1)$ on average." **L5 (Senior):** Given a use case needing millions of appends but rare random access, justify whether `ArrayList` is still the right choice or whether `LinkedList` (Ch.3) might be better. **L6 (Mastery):** Explain, from memory, exactly what happens internally during an `ArrayList` resize.

CHAPTER 3 — LinkedList, Vector & Stack

Telugu Explanation

`LinkedList` అనేది doubly-linked list backed `List` (మరియు `Deque`) implementation — ప్రతి node కి `prev / next` references ఉంటాయి, index-based access కి $O(n)$ కానీ beginning/end insertion/removal కి $O(1)$. `Vector` `ArrayList` లాంటిదే కానీ **synchronized** (legacy, thread-safe కానీ slow). `Stack` `Vector` ని extend చేస్తుంది, LIFO operations కోసం — ఇది కూడా legacy, ఆధునిక code `Deque` వాడాలి.

Professional English Explanation

`LinkedList` is a doubly-linked-list-based implementation of both `List` and `Deque` — each node holds `prev / next` references, giving $O(1)$ insertion/removal at either end but $O(n)$ indexed access (must walk the list). `Vector` is a legacy, synchronized analog of `ArrayList` (thread-safe but with synchronization overhead on every call, even single-threaded use). `Stack` extends `Vector` for LIFO operations — legacy and generally discouraged in favor of `ArrayDeque` (Ch.9) for stack-like usage.

Diagram — LinkedList Node Structure

```
null <- [prev|data=A|next] <-> [prev|data=B|next] <-> [prev|data=C|next] -> null
           ^ head                                     ^ tail
Insertion/removal at head or tail:  $O(1)$  - just relink a few pointers
Access by index (e.g., get(1)):  $O(n)$  - must walk from head OR tail (whichever is closer)
```

Java Code

```
import java.util.*;

public class LinkedListVectorStackDemo {
    public static void main(String[] args) {
        LinkedList<String> linked = new LinkedList<>();
        linked.addFirst("B");
        linked.addFirst("A");           //  $O(1)$  - just relinks head pointer
        linked.addLast("C");            //  $O(1)$  - just relinks tail pointer
        System.out.println("LinkedList: " + linked);
        System.out.println("First: " + linked.getFirst() + ", Last: " + linked.getLast());

        Vector<Integer> vector = new Vector<>();           // legacy, synchronized
        vector.add(1); vector.add(2);
        System.out.println("Vector: " + vector);

        Stack<Integer> stack = new Stack<>();             // legacy LIFO, extends Vector
        stack.push(10); stack.push(20); stack.push(30);
        System.out.println("Stack pop: " + stack.pop());  // removes and returns 30 (LIFO)
        System.out.println("Stack after pop: " + stack);

        // Modern idiomatic replacement for Stack: ArrayDeque (Ch.9)
        Deque<Integer> modernStack = new ArrayDeque<>();
        modernStack.push(10); modernStack.push(20);
        System.out.println("ArrayDeque as stack, pop: " + modernStack.pop());
    }
}
```

Output

```
LinkedList: [A, B, C]
First: A, Last: C
Vector: [1, 2]
Stack pop: 30
Stack after pop: [10, 20]
ArrayDeque as stack, pop: 20
```

Internal Working & Complexity

Structure	get(index)	add/remove at end	add/remove at beginning	Thread-safe?
ArrayList	O(1)	Amortized O(1)	O(n)	No
LinkedList	O(n)	O(1)	O(1)	No
Vector	O(1)	Amortized O(1), synchronized	O(n), synchronized	Yes (legacy, coarse locking)
Stack (extends Vector)	O(1)	O(1) push/pop at end	—	Yes (legacy, coarse locking)

- `LinkedList` 's O(n) indexed access is a genuine practical downside — even though it "feels" like it should be flexible, iterating it via index (`for (int i...) list.get(i)`) is a classic performance bug (O(n²) overall), while iterating with an `Iterator` /for-each is O(n) total, since the iterator walks node-to-node without re-searching from the head each time.
- `Vector` / `Stack` 's synchronization is **coarse-grained** (every method call is synchronized individually) — this doesn't even guarantee compound operations (check-then-act) are atomic, making it a poor fit for real concurrent correctness despite being "thread-safe" in a narrow sense (Book 08 covers proper concurrent collections).

Real-World Example

Telugu: LRU cache implementations, undo/redo functionality, DFS-style algorithms లో

`Deque` / `LinkedList` వాడతారు — end-based insertion/removal frequent గా జరిగే scenarios కి ఇవి సరిపోతాయి. `Vector` / `Stack` ఆధునిక codebases లో దాదాపు ఎప్పుడూ కనిపించవు — legacy code maintain చేసేటప్పుడు మాత్రమే చూస్తారు. English: `Deque` / `LinkedList` -style structures fit LRU caches, undo/redo stacks, and DFS-style algorithms where frequent insertion/removal at the ends matters; `Vector` / `Stack` are essentially legacy-only in modern codebases — you'll mostly encounter them maintaining older code, not writing new code.

Interview Answer

"`LinkedList` is a doubly-linked list giving O(1) insertion/removal at either end but O(n) indexed access; `ArrayList` is the opposite trade-off. `Vector` and `Stack` are legacy synchronized collections — thread-safe in a narrow, coarse-grained sense, but generally replaced in modern code by `ArrayDeque` or proper concurrent collections (Book 08)."

Cross Questions

- Q: Why is iterating a `LinkedList` by index in a `for` loop a performance bug? → A: Each `get(i)` call walks from the head (or tail) to reach index `i`, making a full index-based traversal $O(n^2)$ overall instead of $O(n)$.
- Q: Why is `Vector` considered legacy despite being thread-safe? → A: Its synchronization is per-method, which adds overhead even in single-threaded use and still doesn't make compound operations (like check-then-add) atomic — modern code prefers unsynchronized collections plus explicit synchronization, or proper concurrent collections, based on actual needs (Book 08).
- Q: What's the modern replacement for `Stack`? → A: `ArrayDeque`, used via `push()` / `pop()` — faster (no synchronization overhead) and it's the JDK-documented recommended replacement.

Tricky Questions

- Q: Does `LinkedList` implement `Deque`? → A: Yes — it implements both `List` and `Deque`, making it usable as a stack, queue, or double-ended queue, though `ArrayDeque` is generally preferred for pure stack/queue use due to better cache locality and no per-node object overhead.
- Q: Is a `LinkedList` always slower than an `ArrayList`? → A: No — for workloads dominated by insertions/removals at the ends (not indexed access or `contains()`), `LinkedList` can genuinely outperform `ArrayList`'s $O(n)$ shifting cost.

Coding Exercise

L1: Build a `LinkedList`, use `addFirst` / `addLast` / `removeFirst` / `removeLast`, and print after each operation. **L2:** Reproduce the $O(n^2)$ index-based `LinkedList` iteration bug and fix it using an enhanced for-loop. **L3:** Rewrite a `Stack`-based LIFO algorithm (e.g., balanced-parentheses checker) using `ArrayDeque` instead. **L4 (Interview):** Explain the get/add/remove complexity trade-off between `ArrayList` and `LinkedList`. **L5 (Senior):** Given an LRU cache requirement, decide between `LinkedList + HashMap` (manual) vs `LinkedHashMap` (Ch.7) and justify your choice. **L6 (Mastery):** Draw the doubly-linked node diagram from memory and explain why head/tail operations are $O(1)$.

CHAPTER 4 — HashSet Deep Dive

Telugu Explanation

HashSet duplicates allow చేయని, **ordering guarantee లేని** collection. Internally HashSet ఒక HashMap ని wrap చేస్తుంది — ప్రతి element HashMap లో key గా store అవుతుంది, value ఒక dummy constant object. కాబట్టి HashSet యొక్క performance పూర్తిగా HashMap 's hashing mechanics మీద ఆధారపడి ఉంటుంది (Ch.6 లో లోతుగా).

Professional English Explanation

HashSet disallows duplicates and provides no ordering guarantee. Internally, HashSet is literally backed by a `HashMap<E, Object>` — each element is stored as a **key** in that internal map, with a shared dummy constant as the value. This means HashSet 's performance characteristics are entirely inherited from HashMap 's hashing mechanics (fully detailed in Ch.6).

Java Code

```
import java.util.*;

public class HashSetDemo {
    static class Point {
        int x, y;
        Point(int x, int y) { this.x = x; this.y = y; }
        @Override public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Point p)) return false;
            return x == p.x && y == p.y;
        }
        @Override public int hashCode() { return Objects.hash(x, y); }
        @Override public String toString() { return "(" + x + "," + y + ")"; }
    }

    public static void main(String[] args) {
        Set<String> set = new HashSet<>();
        set.add("apple"); set.add("banana"); set.add("apple"); // duplicate silently
        // ignored
        System.out.println("Set (duplicates removed): " + set + ", size=" + set.size());

        Set<Point> points = new HashSet<>();
        points.add(new Point(1, 2));
        points.add(new Point(1, 2)); // "equal" content ->
        // treated as duplicate
        System.out.println("Points set size (equals/hashCode overridden): " + points.size());

        System.out.println("Contains (1,2)? " + points.contains(new Point(1, 2))); // true -
        // relies on equals/hashCode
    }
}
```

Output (illustrative — HashSet iteration order isn't guaranteed)

```
Set (duplicates removed): [banana, apple], size=2
Points set size (equals/hashCode overridden): 1
Contains (1,2)? true
```

Internal Working — Why `equals()` / `hashCode()` Matter Here

- `HashSet.add(e)` internally computes `e.hashCode()`, uses it to locate a **bucket** (a slot in the backing array), then checks `equals()` against existing entries in that bucket to detect duplicates — this is exactly the equals-hashCode contract from Book 01, Ch.15 and Book 02, in direct, practical action.
- If `Point` did **not** override `equals()` / `hashCode()`, new `Point(1,2)` and another new `Point(1,2)` would use `Object`'s default identity-based versions — two *different* objects would never be considered duplicates, and `points.size()` would incorrectly be `2`.
- `HashSet` guarantees $O(1)$ average-case `add` / `remove` / `contains` — entirely because `HashMap` (Ch.6) provides that guarantee; a poorly-distributed `hashCode()` (e.g., always returning the same constant) would degrade this to $O(n)$ by forcing everything into one bucket.

Real-World Example

Telugu: Duplicate `Employee` records detect చేయడానికి `HashSet<Employee>` వాడేటప్పుడు, `Employee` class లో `equals()` / `hashCode()` (business key, ఉదా. `employeeId` మీద ఆధారపడి) override చేయకపోతే, duplicates సరిగ్గా detect అవ్వవు — ఇది Book 02లో మనం చూసిన real bug pattern నే. English: Detecting duplicate `Employee` records via `HashSet<Employee>` silently fails to work correctly unless `equals()` / `hashCode()` are overridden by business key (e.g., `employeeId`) — the exact same real bug pattern first flagged in Book 02, now shown in its most common practical context: collections.

Interview Answer

"`HashSet` is backed internally by a `HashMap`, storing each element as a key with a dummy value — so its $O(1)$ average performance and duplicate-detection behavior depend entirely on correct `equals()` / `hashCode()` implementations on the stored elements, exactly per the equals-hashCode contract."

Cross Questions

- Q: Does `HashSet` maintain insertion order? → A: No — iteration order is unspecified and can appear to change across JVM versions/hash distributions; use `LinkedHashSet` (Ch.5) if insertion order matters.
- Q: What happens if you add a mutable object to a `HashSet`, then mutate a field used in its `hashCode()`? → A: The object becomes "lost" — its bucket position was computed from the old hash; `contains()` / `remove()` using an equal key will likely fail to find it (the exact bug previewed in Book 01, Ch.15).
- Q: Is `HashSet` thread-safe? → A: No — use `Collections.synchronizedSet()` or `ConcurrentHashMap.newKeySet()` (Book 08) for concurrent access.

Tricky Questions

- Q: If two objects have the same `hashCode()` but `equals()` returns `false`, does `HashSet` still store both? → A: Yes — this is a legal hash collision (Book 01, Ch.15); both objects land in the same bucket but are correctly distinguished by `equals()` and both are kept.
- Q: Can `null` be added to a `HashSet`? → A: Yes — exactly one `null` element is allowed (since it's backed by `HashMap`, which permits one `null` key).

Coding Exercise

L1: Add duplicate `String` `s` to a `HashSet` and confirm only unique values remain. **L2:** Reproduce the "no equals/hashCode override" bug with a custom class, then fix it. **L3:** Reproduce the "mutable hashCode field" bug (add an object, mutate its hash-relevant field, then fail to find it via `contains()`). **L4 (Interview):** Explain exactly how `HashSet.add()` uses `hashCode()` and `equals()` together. **L5 (Senior):** Design a duplicate-detection service for incoming event records, deciding what business key to use for `equals()` / `hashCode()` and why immutability (Book 01, Ch.15) matters for objects used this way. **L6 (Mastery):** Explain, from memory, why `HashSet` is "just a `HashMap` in disguise."

CHAPTER 5 — LinkedHashSet & TreeSet

Telugu Explanation

`LinkedHashSet` `HashSet` లాగే పనిచేస్తుంది కానీ **insertion order** maintain చేస్తుంది (internally ఒక doubly-linked list అదనంగా వాడుతుంది, bucket structure మేరీద). `TreeSet` elements ని **sorted order** లో maintain చేస్తుంది (Red-Black Tree ఆధారంగా), `Comparable` / `Comparator` (Ch.10) వాడి.

Professional English Explanation

`LinkedHashSet` behaves like `HashSet` but preserves **insertion order**, by maintaining an additional internal doubly-linked list threading through the hash buckets. `TreeSet` maintains elements in **sorted order** (natural ordering via `Comparable`, or a supplied `Comparator`, Ch.10), implemented internally as a **Red-Black Tree** — giving $O(\log n)$ operations instead of $O(1)$, in exchange for always-sorted iteration.

Java Code

```
import java.util.*;

public class OrderedSetsDemo {
    public static void main(String[] args) {
        Set<String> hashSet = new HashSet<>(List.of("banana", "apple", "cherry"));
        Set<String> linkedHashSet = new LinkedHashSet<>(List.of("banana", "apple", "cherry"));
        Set<String> treeSet = new TreeSet<>(List.of("banana", "apple", "cherry"));

        System.out.println("HashSet (unspecified order): " + hashSet);
        System.out.println("LinkedHashSet (insertion order): " + linkedHashSet);
        System.out.println("TreeSet (sorted order): " + treeSet);

        TreeSet<Integer> numbers = new TreeSet<>(List.of(50, 10, 40, 20, 30));
        System.out.println("First: " + numbers.first() + ", Last: " + numbers.last());
        System.out.println("HeadSet(30): " + numbers.headSet(30));           // elements < 30
        System.out.println("TailSet(30): " + numbers.tailSet(30));           // elements >= 30
        System.out.println("Ceiling(25): " + numbers.ceiling(25));           // smallest element
        // >= 25
        System.out.println("Floor(25): " + numbers.floor(25));               // largest element <=
        // 25
    }
}
```

Output

```
HashSet (unspecified order): [banana, cherry, apple]
LinkedHashSet (insertion order): [banana, apple, cherry]
TreeSet (sorted order): [apple, banana, cherry]
First: 10, Last: 50
HeadSet(30): [10, 20]
TailSet(30): [30, 40, 50]
Ceiling(25): 30
Floor(25): 20
```

Internal Working

- `LinkedHashSet` pays a small extra memory/pointer-maintenance cost over `HashSet` for its ordering guarantee, but retains the same $O(1)$ average `add` / `contains` / `remove` — it's `HashMap`'s lesser-known cousin, `LinkedHashMap` (Ch.7), underneath.
- `TreeSet` requires elements to be mutually comparable — either they implement `Comparable` (Ch.10) or a `Comparator` is supplied at construction — and throws `ClassCastException` at runtime if incomparable elements are added, since the Red-Black Tree needs a total ordering to place every element correctly.
- `TreeSet`'s $O(\log n)$ operations are the direct cost of maintaining sorted order — this is a genuine, deliberate trade-off versus `HashSet`'s $O(1)$, not a flaw.

Real-World Example

Telugu: Leaderboard systems (sorted scores), price range queries
(`headSet` / `tailSet` / `ceiling` / `floor`) — ఇవి `TreeSet` కి ideal use cases; UI dropdown లో insertion order maintain చేయాలంటే `LinkedHashSet`. English: Leaderboards (always-sorted scores) and range queries (`headSet` / `tailSet` / `ceiling` / `floor` for "closest match" lookups) are ideal `TreeSet` use cases; preserving the exact order items were added (e.g., a UI dropdown built from a set of distinct recently-viewed items) is the classic `LinkedHashSet` use case.

Interview Answer

"`LinkedHashSet` preserves insertion order at the same average $O(1)$ cost as `HashSet`, using an internal linked list alongside the hash buckets. `TreeSet` maintains sorted order via a Red-Black Tree, giving $O(\log n)$ operations in exchange for always-sorted iteration and rich range-query methods like `headSet` / `tailSet` / `ceiling` / `floor`."

Cross Questions

- Q: Why does `TreeSet` need `Comparable` or a `Comparator`? → A: A Red-Black Tree requires a total ordering to decide where each new element belongs relative to existing ones — without one, it can't maintain sorted structure.
- Q: What happens if you add an element to a `TreeSet` that can't be compared to existing elements? → A: `ClassCastException` at runtime, at the moment of insertion.
- Q: Is `LinkedHashSet`'s ordering the same as `TreeSet`'s? → A: No — `LinkedHashSet` preserves *insertion* order; `TreeSet` maintains *sorted* order, which can be very different depending on insertion sequence.

Tricky Questions

- Q: Can a `TreeSet` contain `null`? → A: No (in most cases) — comparing `null` against existing elements throws `NullPointerException`, unlike `HashSet` / `LinkedHashSet` which permit one `null`.
- Q: Does `LinkedHashSet` support access-order (like some `LinkedHashMap` configurations, Ch.7)? → A: No — `LinkedHashSet` only supports insertion-order; access-order reordering is a `LinkedHashMap`-specific feature.

Coding Exercise

L1: Insert the same elements into `HashSet`, `LinkedHashSet`, and `TreeSet`, and compare their iteration order. **L2:** Use `TreeSet`'s `ceiling` / `floor` / `higher` / `lower` methods to implement a "closest price match" lookup. **L3:** Reproduce the `ClassCastException` from adding incomparable custom objects to a `TreeSet` without a `Comparator`. **L4 (Interview):** Explain why `TreeSet` operations are $O(\log n)$ instead of $O(1)$. **L5 (Senior):** Design a leaderboard using `TreeSet<Player>` with a custom `Comparator` sorting by score descending, then name/ID as a tiebreaker. **L6 (Mastery):** Explain, from memory, the internal structural difference between `HashSet`, `LinkedHashSet`, and `TreeSet`.

Java Code

```
import java.util.*;

public class HashMapInternalsDemo {
    static class BadKey {
        int id;
        BadKey(int id) { this.id = id; }
        @Override public int hashCode() { return 1; }           // deliberately terrible -
forces all collisions
        @Override public boolean equals(Object o) { return o instanceof BadKey b && b.id == id; }
    }

    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("cat", 1);
        map.put("dog", 2);
        map.put("cat", 100);                                // same key - REPLACES value, doesn't add a
new entry
        System.out.println("Map: " + map + ", size=" + map.size());

        System.out.println("get('cat'): " + map.get("cat"));
        System.out.println("get('missing'): " + map.get("missing"));    // null - no
exception

        // Demonstrate a deliberately bad hashCode causing full collision (all in ONE bucket)
        Map<BadKey, String> badMap = new HashMap<>();
        for (int i = 0; i < 5; i++) badMap.put(new BadKey(i), "value" + i);
        System.out.println("BadKey map size (still correct via equals): " + badMap.size());
        // Internally: all 5 entries chain in the SAME bucket -> O(n) lookups instead of O(1)!
    }
}
```

Output

```
Map: {cat=100, dog=2}, size=2
get('cat'): 100
get('missing'): null
BadKey map size (still correct via equals): 5
```

Internal Working — The Full `put()` Algorithm

1. Compute `hash(key)` — HotSpot's `HashMap` XORs the hash code with its own bits shifted right 16 ($h \oplus (h \gg 16)$) to "spread" high-order bits into the low-order bits, improving distribution for keys whose `hashCode()` differs mainly in high bits.
2. Compute bucket index: `(table.length - 1) & hash` — a fast bitwise AND, which works because table length is always a power of 2.
3. If the bucket is empty, insert directly.
4. If not, walk the chain (or tree): if a node with an `equals()` -matching key is found, **replace** its value; otherwise, append a new node.
5. If chain length reaches 8 **and** table capacity is at least 64, **treeify** that bucket into a Red-Black Tree.

6. If total size now exceeds `capacity × loadFactor`, **resize**: double the table, and rehash/redistribute every existing entry into the new table.

Real-World Example

Telugu: `hashCode()` బాగా design చేయకపోతే (అన్ని objects ఒకే hash return చేస్తే), `HashMap` $O(1)$ కాకుండా $O(n)$ గా degrade అవుతుంది — ఇది production performance bug కి direct root cause అవుతుంది, ముఖ్యంగా large maps తో. English: A poorly-designed `hashCode()` (e.g., always returning a constant) silently degrades every `HashMap` operation from $O(1)$ average to $O(n)$ — this is a real, subtle production performance bug, especially painful at scale (millions of entries), and one Java 8's treeification specifically mitigates (capping worst-case at $O(\log n)$ instead of $O(n)$), though good `hashCode` design is still the real fix).

Interview Answer

"`HashMap` uses an array of buckets selected by a spread function applied to the key's `hashCode()`. Collisions chain as a linked list, treeified into a Red-Black Tree since Java 8 once a bucket reaches 8 entries (with table capacity ≥ 64), capping worst-case lookup at $O(\log n)$ instead of $O(n)$. The table resizes (doubles) and rehashes everything once entries exceed capacity times the load factor, default 0.75."

Deep Interview Answer

"The hash-spreading step $(h \wedge (h \ggg 16))$ exists specifically because bucket selection only uses the low bits of the hash $((table.length - 1) \& hash)$, valid since capacity is always a power of 2) — without spreading, two hash codes differing only in high bits would collide even with a perfectly reasonable `hashCode()` implementation. Treeification (Java 8) is a defensive worst-case improvement, not a substitute for good `hashCode()` design — it protects against pathological/adversarial collision patterns (a known category of denial-of-service attack against naive hash tables), but a well-distributed `hashCode()` keeps operations at $O(1)$ average in the first place, which treeification doesn't improve on."

Cross Questions

- Q: Why is the default load factor 0.75, not 1.0? → A: It's a space/time trade-off — 0.75 keeps average chain length short (fast lookups) while not wasting too much unused array capacity; 1.0 would pack the table tighter but increase collision frequency, degrading performance.
- Q: Why must `HashMap`'s table size always be a power of 2? → A: So bucket index computation can use a fast bitwise AND $((capacity - 1) \& hash)$ instead of a slower modulo operation, and so that on resize (always doubling), each entry's new bucket is deterministically either its old index or old index + old capacity.
- Q: Does treeification change the `Map` contract or iteration semantics? → A: No — it's purely an internal implementation optimization for that bucket's *storage*; `get` / `put` / `entrySet` behavior and contracts remain unchanged.

Tricky Questions

- Q: If `capacity` doubles during a resize, does EVERY entry necessarily move to a different bucket? → A: No — HotSpot's resize algorithm cleverly determines each entry either stays at the same index or moves to `oldIndex + oldCapacity`, based on one additional hash bit, avoiding a full recomputation of every hash.
- Q: Can two keys with different `hashCode()` values ever end up in the same bucket? → A: Yes — bucket index is `hash & (capacity-1)`, so any two hash codes sharing the same low bits (post-spreading) map to the same bucket, even if the full hash codes differ.

Coding Exercise

L1: Insert 20 entries into a `HashMap` and explain, conceptually, when a resize would have been triggered (default capacity 16, load factor 0.75). **L2:** Reproduce the `BadKey` all-same-`hashCode()` example and reflect on why lookups degrade. **L3:** Implement a simple `hashCode()` for a custom class using `Objects.hash(...)` and verify good distribution by printing hash values for 10 different instances. **L4 (Interview):** Walk through the full `put()` algorithm step by step, from `hashCode()` call to final storage. **L5 (Senior):** Explain treeification's purpose as a defensive measure against pathological hash collisions (including the DoS-attack angle) and why it doesn't replace the need for good `hashCode()` design. **L6 (Mastery):** Explain, from memory, why table capacity is always a power of 2 and how that enables fast bucket-index computation.

CHAPTER 7 — LinkedHashMap, TreeMap & Hashtable

Telugu Explanation

`LinkedHashMap` `HashMap` లాగే పనిచేస్తుంది కానీ insertion order (లేదా, configure చేస్తే, **access order**) maintain చేస్తుంది — LRU cache implement చేయడానికి ఇది ఖచ్చితంగా సరిపోతుంది. `TreeMap` keys ని sorted order లో maintain చేస్తుంది (Red-Black Tree, `TreeSet` లాగే). `Hashtable` `HashMap` కి పాత, synchronized, legacy alternative — `null` keys/values allow చేయదు.

Professional English Explanation

`LinkedHashMap` behaves like `HashMap` but maintains insertion order (or, when configured, **access order** — reordering entries to most-recently-accessed-last), making it the textbook building block for an **LRU cache** via its overridable `removeEldestEntry()` hook. `TreeMap` maintains keys in sorted order, implemented as a Red-Black Tree, mirroring `TreeSet`'s internals. `Hashtable` is a legacy, synchronized alternative to `HashMap` that disallows `null` keys/values — largely superseded by `ConcurrentHashMap` (Ch.8) in modern code.

Java Code — LinkedHashMap as an LRU Cache

```
import java.util.*;

public class OrderedMapsDemo {

    static class LruCache<K, V> extends LinkedHashMap<K, V> {
        private final int capacity;
        LruCache(int capacity) {
            super(16, 0.75f, true);          // 'true' = access-order mode, not insertion-order
            this.capacity = capacity;
        }
        @Override
        protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
            return size() > capacity;         // auto-evict the least-recently-used entry
        }
    }

    public static void main(String[] args) {
        LruCache<Integer, String> cache = new LruCache<>(3);
        cache.put(1, "A"); cache.put(2, "B"); cache.put(3, "C");
        cache.get(1);           // accessing 1 marks it "recently used"
        cache.put(4, "D");       // capacity exceeded -> evicts least-
recently-used (2, "B")

        System.out.println("LRU cache after eviction: " + cache);

        TreeMap<String, Integer> sortedAges = new TreeMap<>();
        sortedAges.put("Charlie", 35); sortedAges.put("Alice", 28); sortedAges.put("Bob", 30);
        System.out.println("TreeMap (sorted by key): " + sortedAges);
        System.out.println("First entry: " + sortedAges.firstEntry());

        Hashtable<String, Integer> legacy = new Hashtable<>();
        legacy.put("x", 1);
        try {
            legacy.put(null, 99);           // throws NullPointerException - Hashtable disallows
null keys/values
        } catch (NullPointerException e) {
            System.out.println("Hashtable rejects null: " + e.getClass().getSimpleName());
        }
    }
}
```

Output

```
LRU cache after eviction: {3=C, 1=A, 4=D}
TreeMap (sorted by key): {Alice=28, Bob=30, Charlie=35}
First entry: Alice=28
Hashtable rejects null: NullPointerException
```

Internal Working

- `LinkedHashMap` 's access-order mode (`new LinkedHashMap<>(initialCapacity, loadFactor, true)`) re-links an entry to the tail of its internal doubly-linked list every time it's accessed via `get()` — combined with overriding `removeEldestEntry()`, this is the **entire implementation** of a correct, simple LRU cache in a few lines, no manual list/map coordination needed.

- `TreeMap` requires keys to be mutually comparable, exactly like `TreeSet` (Ch.5) — same `ClassCastException` risk for incomparable types.
- `Hashtable` predates the Collections Framework (it's from Java 1.0) and was retrofitted to implement `Map` later — its `null`-rejection and full-method synchronization are both historical artifacts; `ConcurrentHashMap` (Ch.8) is the modern, better-performing choice for concurrent needs.

Real-World Example

Telugu: In-memory LRU cache (session cache, recently-viewed products) implement చేయడానికి `LinkedHashMap` access-order mode అత్యంత సులభమైన, correct పద్ధతి. `TreeMap` alphabetically sorted reports/dropdowns కి వాడతారు. English: An access-order `LinkedHashMap` is the simplest correct way to implement an in-memory LRU cache (session caches, "recently viewed products") without hand-rolling list/map coordination; `TreeMap` is the natural fit whenever a sorted-by-key view (alphabetical reports, sorted dropdowns) is required directly from the map itself.

Interview Answer

"`LinkedHashMap` adds insertion-order (or optionally access-order) iteration on top of `HashMap`'s hashing, and its `removeEldestEntry()` hook makes it the standard building block for LRU caches. `TreeMap` keeps keys sorted via a Red-Black Tree. `Hashtable` is a legacy synchronized `Map` implementation, largely superseded by `ConcurrentHashMap`."

Cross Questions

- Q: How does access-order `LinkedHashMap` know when to evict? → A: It doesn't automatically — you must override `removeEldestEntry()` to return `true` under your eviction condition (e.g., size exceeding a capacity); by default it always returns `false` (never auto-evicts).
- Q: Can `TreeMap` be constructed with a custom `Comparator`? → A: Yes — `new TreeMap<>(comparator)`, exactly like `TreeSet` (Ch.5, Ch.10).
- Q: Why does `Hashtable` disallow `null` keys and values while `HashMap` allows one `null` key? → A: A design decision from `Hashtable`'s original (pre-Collections-Framework) API — `HashMap` was designed later with more permissive semantics; this remains a key practical difference to know.

Tricky Questions

- Q: Does `LinkedHashMap` in default (insertion-order) mode reorder entries on `get()`? → A: No — only access-order mode (constructed with the third `boolean` argument `true`) reorders on access; default mode strictly preserves insertion order regardless of access pattern.
- Q: Is `Hashtable` a legitimate choice for new code needing thread safety? → A: Essentially never in modern code — `ConcurrentHashMap` (Ch.8) offers far better concurrent performance with finer-grained locking and equivalent (or better) safety guarantees.

Coding Exercise

L1: Build a `LinkedHashMap` in default (insertion-order) mode and confirm `get()` does NOT reorder it. **L2:** Implement the LRU cache example yourself, testing eviction with a capacity of 2. **L3:** Build a

`TreeMap` with a custom `Comparator` sorting entries by value instead of key (hint: this requires extracting entries into a list first, since `TreeMap` itself sorts by key). **L4 (Interview):** Explain exactly how `removeEldestEntry()` enables LRU cache behavior. **L5 (Senior):** Design a bounded, thread-unsafe-but-documented LRU cache class wrapping `LinkedHashMap`, and note (as a comment) what would need to change for thread-safety (bridges to Book 08). **L6 (Mastery):** Explain, from memory, the difference between `LinkedHashMap`'s insertion-order and access-order modes.

CHAPTER 8 — ConcurrentHashMap: Structural Overview

Telugu Explanation

`ConcurrentHashMap` multiple threads ఏకకాలంలో safely access చేయగలిగే Map — `Hashtable` లా పూర్తి map ని lock చేయకుండా, **finer-grained locking** (bucket-level, Java 8+ లో CAS operations + node-level synchronization) వాడుతుంది, ఇది చాలా better concurrency throughput ఇస్తుంది. దీని పూర్తి concurrency mechanics **Book 08** లో వివరంగా.

Professional English Explanation

`ConcurrentHashMap` allows safe concurrent access from multiple threads without locking the entire map like `Hashtable` / `Collections.synchronizedMap()` do. Since Java 8, it uses a combination of CAS (Compare-And-Swap) operations for uncontended bucket updates and fine-grained per-bucket (node-level) synchronization only when actually needed — dramatically improving concurrent throughput. Full concurrency mechanics (CAS, locks, memory visibility) are covered in **Book 08**; this chapter gives the structural picture from a Collections perspective.

Java Code

```
import java.util.concurrent.*;
import java.util.*;

public class ConcurrentHashMapDemo {
    public static void main(String[] args) throws InterruptedException {
        Map<String, Integer> unsafe = new HashMap<>();
        Map<String, Integer> concurrent = new ConcurrentHashMap<>();

        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) {
                concurrent.merge("counter", 1, Integer::sum);    // atomic, thread-safe
                // compound update
            }
        };

        Thread t1 = new Thread(task), t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("ConcurrentHashMap counter (expected 2000): " +
            concurrent.get("counter"));

        // Comparison: a plain HashMap under the same concurrent access has UNDEFINED behavior
        // (data loss, infinite loops in old Java 7 resize bugs, or exceptions) - never do this
        // in production.
        System.out.println("Structural comparison only - do not run concurrent writes on a
            plain HashMap.");
    }
}
```

Output

```
ConcurrentHashMap counter (expected 2000): 2000
Structural comparison only - do not run concurrent writes on a plain HashMap.
```

Internal Working (Structural Summary — Full Detail in Book 08)

- **Pre-Java 8:** used **segment-based locking** — the table was divided into a fixed number of "segments," each independently lockable, so different threads writing to different segments didn't block each other.
- **Java 8+:** segments were removed in favor of locking **individual bin (bucket) head nodes** only when a write actually contends, combined with CAS for the common uncontended case — finer-grained and more efficient than the old segment approach.
- Read operations (`get()`) are largely **lock-free**, relying on `volatile` reads (Book 08's Java Memory Model) to see recent writes without blocking.
- Compound atomic operations like `merge()` , `computeIfAbsent()` , `putIfAbsent()` are the idiomatic, safe way to perform "read-then-write" logic on a `ConcurrentHashMap` — using separate `get()` + `put()` calls reintroduces race conditions even on a concurrent map.

Real-World Example

Telugu: Multi-threaded web servers లో shared cache (request counts, session data)

`ConcurrentHashMap` వాడి safely manage చేస్తారు — `HashMap` వాడితే concurrent modification వల్ల data corruption లేదా infinite loops (పాత Java versions లో) వచ్చే ప్రమాదం ఉంది. English: Multi-threaded web servers use `ConcurrentHashMap` for shared caches (request counters, session data) precisely to avoid the real, historically documented danger of using a plain `HashMap` under concurrent writes — including a notorious old bug where concurrent resizing could corrupt the internal linked list into a cycle, causing `get()` to loop forever.

Interview Answer

"`ConcurrentHashMap` allows safe, high-throughput concurrent access without locking the whole map — modern versions (Java 8+) use per-bucket locking only on contention plus CAS operations for the common case, replacing the older segment-locking design. It's the standard modern replacement for `Hashtable` / `Collections.synchronizedMap()` in concurrent code. Full concurrency internals are in Book 08."

Cross Questions

- Q: Why is using separate `get()` then `put()` on a `ConcurrentHashMap` still potentially unsafe for compound logic? → A: Even though each individual call is thread-safe, another thread can modify the value between your `get()` and `put()` — atomic compound methods (`merge` , `computeIfAbsent` , `putIfAbsent`) close this gap.
- Q: Does `ConcurrentHashMap` allow `null` keys or values? → A: No — unlike `HashMap` , it disallows both, specifically because `null` is ambiguous in a concurrent context (you can't distinguish "key

absent" from "key present with null value" safely when another thread might be concurrently modifying the map).

- Q: How did the locking strategy change between pre-Java-8 and Java-8+ `ConcurrentHashMap`? → A: Pre-8 used a fixed number of lockable "segments"; Java 8+ removed segments in favor of per-bucket-node locking plus CAS, generally improving concurrency further.

Tricky Questions

- Q: Is `ConcurrentHashMap.size()` guaranteed to be perfectly accurate at the instant it's called under concurrent modification? → A: It's a best-effort estimate in highly concurrent scenarios (though quite accurate in practice) — the strict contract prioritizes not blocking other operations over perfect real-time accuracy.
- Q: Why does `ConcurrentHashMap` disallow `null` while `HashMap` allows one `null` key? → A: In a single-threaded `HashMap`, `map.get(key) == null` unambiguously means "absent" (since you control all access); in a concurrent map, that same check is racy — another thread could have just removed the key — so the API removes the ambiguity by disallowing `null` entirely, forcing use of `containsKey()` or `computeIfAbsent` idioms instead.

Coding Exercise

L1: Run the counter demo, then rerun it using a plain `HashMap` and a non-atomic `get + put` pattern instead of `merge()` — observe the (likely) undercounting due to a race condition. **L2:** Use `computeIfAbsent()` on a `ConcurrentHashMap` to implement a thread-safe memoizing cache. **L3:** Research (comment-only, no need to reproduce) the classic Java 7 `HashMap` concurrent-resize infinite-loop bug and summarize it in your own words. **L4 (Interview):** Explain why `ConcurrentHashMap` disallows `null` keys/values while `HashMap` allows a `null` key. **L5 (Senior):** Design a thread-safe request-counting middleware using `ConcurrentHashMap.merge()`, explaining why it's safe under concurrent load. **L6 (Mastery):** Explain, from memory, the structural evolution from segment-locking to per-bucket-locking+CAS across Java versions (full mechanics deferred to Book 08).

CHAPTER 9 — Queue & Deque: PriorityQueue, ArrayDeque

Telugu Explanation

`Queue` FIFO (First-In-First-Out) processing కోసం design చేయబడింది. `Deque` (Double-Ended Queue) రెండు చివరలా (head మరియు tail) insertion/removal support చేస్తుంది — stack గా కూడా, queue గా కూడా వాడొచ్చు. `PriorityQueue` elements ని **priority order** లో (natural ordering లేదా `Comparator`) process చేస్తుంది, internally ఒక **binary heap** వాడి.

Professional English Explanation

`Queue` models FIFO processing order. `Deque` (Double-Ended Queue) supports insertion/removal at both ends, making it usable as both a stack and a queue — and is the JDK-recommended replacement for both legacy `Stack` and `LinkedList` -as-queue usage in most cases. `PriorityQueue` processes elements in priority order (natural ordering via `Comparable` , or a supplied `Comparator`), implemented internally as a **binary heap** — giving $O(\log n)$ insertion and $O(\log n)$ removal-of-minimum, with $O(1)$ peek-at-minimum.

Java Code

```
import java.util.*;

public class QueueDequeDemo {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>(); // FIFO
        queue.offer("first"); queue.offer("second"); queue.offer("third");
        System.out.println("Poll (FIFO): " + queue.poll()); // removes "first"
        System.out.println("Queue now: " + queue);

        Deque<String> deque = new ArrayDeque<>();
        deque.addFirst("A"); deque.addLast("B"); deque.addFirst("Z");
        System.out.println("Deque: " + deque); // [Z, A, B]

        deque.push("stackTop"); // push = addFirst (stack
behavior)
        System.out.println("After push (as stack): " + deque);
        System.out.println("Pop: " + deque.pop()); // pop = removeFirst

        PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // natural ordering: min-
heap by default
        minHeap.addAll(List.of(50, 10, 40, 20, 30));
        System.out.print("PriorityQueue poll order (min-heap): ");
        while (!minHeap.isEmpty()) System.out.print(minHeap.poll() + " ");
        System.out.println();

        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder()); //
max-heap via Comparator
        maxHeap.addAll(List.of(50, 10, 40, 20, 30));
        System.out.print("PriorityQueue poll order (max-heap): ");
        while (!maxHeap.isEmpty()) System.out.print(maxHeap.poll() + " ");
        System.out.println();
    }
}
```

Output

```
Poll (FIFO): first
Queue now: [second, third]
Deque: [Z, A, B]
After push (as stack): [stackTop, Z, A, B]
Pop: stackTop
PriorityQueue poll order (min-heap): 10 20 30 40 50
PriorityQueue poll order (max-heap): 50 40 30 20 10
```

Internal Working

- `ArrayDeque` is backed by a **resizable circular array** (not a linked structure) — it has better cache locality and lower per-element overhead than `LinkedList`, which is exactly why it's the JDK-recommended replacement for both `Stack` and queue-usage of `LinkedList`.
- `PriorityQueue`'s binary heap is stored as a flat array where, for index `i`, children live at `2i+1` and `2i+2` — `poll()` removes the root (min or max, per ordering), moves the last element to the root, then "sifts down" to restore heap order, all in $O(\log n)$.

- **Important interview trap:** `PriorityQueue` 's iterator (via `iterator()` /`toString`) does **not** guarantee sorted order — only repeated `poll()` calls guarantee elements come out in priority order; the underlying array is heap-ordered, not fully sorted.

Real-World Example

Telugu: Task scheduling systems (priority-based job queues), Dijkstra's algorithm (Book 20), "top-K elements" problems — ఇవన్నీ `PriorityQueue` యొక్క classic production/algorithmic use cases. `ArrayDeque` sliding-window algorithms, undo/redo functionality & ideal. English: Priority-based job scheduling, Dijkstra's shortest-path algorithm (Book 20), and "find top-K elements" problems are the classic `PriorityQueue` use cases; `ArrayDeque` is ideal for sliding-window algorithms and undo/redo stacks — both are extremely common in real interview coding rounds, not just production code.

Interview Answer

"`Queue` models FIFO order; `Deque` supports both ends, making it the modern replacement for both `Stack` and queue-style `LinkedList` usage, backed by an efficient resizable circular array in `ArrayDeque`. `PriorityQueue` processes elements in priority order using an internal binary heap, giving $O(\log n)$ insertion/removal but critically does NOT guarantee sorted iteration order — only repeated `poll()` does."

Cross Questions

- Q: Why is `ArrayDeque` generally preferred over `LinkedList` for stack/queue usage? → A: Better cache locality (contiguous array vs scattered nodes) and lower per-element memory overhead (no `prev` / `next` pointer objects per element).
- Q: Does iterating a `PriorityQueue` with a for-each loop return elements in priority order? → A: No — this is a classic trap; only sequential `poll()` calls guarantee priority order, since the internal array is heap-ordered, not fully sorted.
- Q: How would you build a max-heap using `PriorityQueue`, which defaults to min-heap? → A: Supply `Comparator.reverseOrder()` (or an equivalent custom comparator) at construction.

Tricky Questions

- Q: Can `ArrayDeque` hold `null` elements? → A: No — `ArrayDeque` explicitly disallows `null` (unlike `LinkedList`), since `null` is used internally as a sentinel to detect an empty slot in the circular array.
- Q: Is `PriorityQueue` thread-safe? → A: No — `PriorityBlockingQueue` (Book 08) is the concurrent-safe analog.

Coding Exercise

L1: Implement a simple FIFO task queue using `ArrayDeque`, and a LIFO undo stack using the same class. **L2:** Build a min-heap and max-heap `PriorityQueue` for the same data set and compare `poll()` sequences. **L3:** Reproduce the "iteration order isn't sorted" trap: print a `PriorityQueue` directly (via `System.out.println`) versus draining it with `poll()`, and compare. **L4 (Interview):**

Explain why `PriorityQueue` iteration order isn't sorted, using the binary-heap array-index relationship. **L5 (Senior):** Implement a "top-K largest elements" algorithm using a bounded min-heap `PriorityQueue` of size K. **L6 (Mastery):** Explain, from memory, the binary heap's array-based parent/child index relationship and why `poll()` is $O(\log n)$.

CHAPTER 10 — Comparable vs Comparator

Telugu Explanation

`Comparable` అనేది ఒక class తన **natural ordering** define చేసుకోవడానికి implement చేసే interface (`compareTo()` method, ఒక్కటే ఆర్డరింగ్ per class). `Comparator` అనేది class కి బయట, **multiple, flexible ordering strategies** define చేయడానికి వాడే separate interface (`compare()` method) — ఒకే class ని వేర్వేరు ways లో sort చేయాలంటే `Comparator` వాడతారు.

Professional English Explanation

`Comparable` lets a class define its own **natural ordering** by implementing `compareTo()` — a class has exactly one natural ordering. `Comparator` is a separate, external interface (`compare()`) for defining one or more **alternative** ordering strategies without modifying the class itself — essential when you need multiple sort orders, or when sorting a class you don't control (can't add `Comparable` to it).

Java Code

```
import java.util.*;

class Employee implements Comparable<Employee> {           // natural ordering: by ID
    String name; int id; double salary;
    Employee(String name, int id, double salary) { this.name = name; this.id = id; this.salary
= salary; }

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id);        // natural order: ascending by ID
    }

    @Override
    public String toString() { return name + "(id=" + id + ", salary=" + salary + ")"; }
}

public class ComparableComparatorDemo {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>(List.of(
            new Employee("Charlie", 3, 70000),
            new Employee("Alice", 1, 90000),
            new Employee("Bob", 2, 60000)
        ));

        Collections.sort(employees);                        // uses compareTo() - natural
ordering (by ID)
        System.out.println("Natural order (by ID): " + employees);

        employees.sort(Comparator.comparingDouble(e -> e.salary)); // Comparator: by salary
ascending
        System.out.println("By salary ascending: " + employees);

        employees.sort(Comparator.comparingDouble((Employee e) -> e.salary).reversed()); //
descending
        System.out.println("By salary descending: " + employees);

        employees.sort(Comparator.comparing((Employee e) -> e.name)
            .thenComparingDouble(e -> e.salary)); // multi-level sort
        System.out.println("By name, then salary: " + employees);
    }
}
```

Output

```
Natural order (by ID): [Alice(id=1, salary=90000.0), Bob(id=2, salary=60000.0), Charlie(id=3,
salary=70000.0)]
By salary ascending: [Bob(id=2, salary=60000.0), Charlie(id=3, salary=70000.0), Alice(id=1,
salary=90000.0)]
By salary descending: [Alice(id=1, salary=90000.0), Charlie(id=3, salary=70000.0), Bob(id=2,
salary=60000.0)]
By name, then salary: [Alice(id=1, salary=90000.0), Bob(id=2, salary=60000.0), Charlie(id=3,
salary=70000.0)]
```

Comparison Table

Aspect	Comparable	Comparator
Method	<code>compareTo(T o)</code>	<code>compare(T o1, T o2)</code>
Location	Implemented BY the class itself	A separate class/lambda, external to the compared class
Number of orderings	Exactly one (the "natural" order)	Unlimited — define as many as needed
Used by	<code>Collections.sort(list)</code> , <code>TreeSet / TreeMap</code> default ordering	<code>list.sort(comparator)</code> , <code>Collections.sort(list, comparator)</code> , <code>TreeSet / TreeMap</code> with explicit comparator
Modifies the class?	Yes — class must implement the interface	No — works on classes you don't control

Internal Working

- `compareTo()` / `compare()` must return negative/zero/positive to indicate less-than/equal/greater-than — both `Collections.sort()` and `TreeSet / TreeMap` rely on this contract to implement their sorting algorithm (a variant of merge sort for `Collections.sort()` , red-black tree ordering for `TreeMap / TreeSet`).
- The `Comparator` interface's default methods (`reversed()` , `thenComparing()` , static `comparing()` / `comparingDouble()` / `comparingInt()`) are Java 8 additions (fully covered in Book 07) that turned verbose anonymous-class comparators into concise, chainable, composable lambda expressions — a major usability upgrade.
- **Consistency with `equals()`** matters: if `compareTo()` returns 0 for two objects that aren't actually `equals()` -equal, a `TreeSet / TreeMap` will treat them as duplicates (since it uses `compareTo()` , not `equals()` , for uniqueness) — a genuine, documented gotcha ("inconsistent with equals").

Real-World Example

Telugu: `Employee` class కి ఒక్కటే natural ordering (ఉదా. ID) ఉంటుంది, కానీ UI requirements వేర్వేరు sort views (by name, by salary, by department) కావాలంటే — ఒక్కొక్క దానికి వేరే `Comparator` వాడతారు, `Employee` class మార్చకుండా. English: An `Employee` class has exactly one natural ordering (say, by ID), but real UI requirements typically need multiple sort views (by name, by salary, by hire date) — each implemented as a separate `Comparator` , without ever touching the `Employee` class itself.

Interview Answer

"`Comparable` lets a class define its single natural ordering via `compareTo()` . `Comparator` is a separate interface for defining any number of alternative orderings via `compare()` , without modifying the class — essential for multiple sort views or sorting classes you don't own. Both are used by `Collections.sort()` / `List.sort()` and by `TreeSet / TreeMap` for maintaining sorted order."

Cross Questions

- Q: Can a class implement `Comparable` and still be sorted differently elsewhere? → A: Yes — `Comparable` defines only the default/natural order; any specific `sort()` call can still override it with an explicit `Comparator`.
- Q: What happens if `compareTo()` is inconsistent with `equals()`? → A: `TreeSet` / `TreeMap` (which use `compareTo()` / `compare()` for uniqueness, not `equals()` / `hashCode()`) may silently treat genuinely-unequal objects as duplicates — a subtle, documented gotcha worth remembering.
- Q: Is `Comparator.comparing(...).thenComparing(...)` evaluated left to right? → A: Yes — `thenComparing` is only consulted as a tiebreaker when the primary comparator returns 0 (equal), exactly like a SQL multi-column `ORDER BY`.

Tricky Questions

- Q: If `TreeSet<Employee>` is used with the natural ordering from the demo (by ID), and two `Employee` objects have the same ID but different names, does the `TreeSet` store both? → A: No — since `compareTo()` returns 0 for equal IDs, `TreeSet` treats them as duplicates and keeps only the first one inserted, regardless of `equals()` / other field differences — the "inconsistent with equals" gotcha in action.
- Q: Can `Comparator.naturalOrder()` be used on a class that doesn't implement `Comparable`? → A: No — it requires the type parameter to extend `Comparable<T>`; the compiler enforces this via generic bounds (Book 06).

Coding Exercise

L1: Implement `Comparable` on a `Product` class (natural order by price) and sort a list using `Collections.sort()`. **L2:** Add 3 different `Comparator`s for the same `Product` class (by name, by price descending, by category then price). **L3:** Reproduce the "inconsistent with equals" `TreeSet` gotcha with a deliberately mismatched `compareTo()` / `equals()` pair. **L4 (Interview):** Explain the difference between `Comparable` and `Comparator` with a concrete scenario for each. **L5 (Senior):** Design a multi-level sort for an `Order` list (by status priority, then by date descending, then by total amount) using chained `Comparator`s. **L6 (Mastery):** Explain, from memory, why `TreeSet` / `TreeMap` uniqueness is governed by `compareTo()` / `compare()`, not `equals()` / `hashCode()`.

CHAPTER 11 — Iterators: Fail-Fast vs Fail-Safe

Telugu Explanation

Fail-fast iterators (`ArrayList` , `HashMap` , `HashSet` వంటి most standard collections) — iteration మధ్యలో collection structurally modify అయితే (add/remove, `Iterator.remove()` కాకుండా) వెంటనే `ConcurrentModificationException` throw చేస్తాయి, bug ని early గా catch చేయడానికి. **Fail-safe** iterators (`CopyOnWriteArrayList` , `ConcurrentHashMap` వంటివి, Book 08) collection యొక్క **copy** me¹ద (లేదా అంతర్గత గా safe గా) iterate చేస్తాయి, exception throw చేయవు.

Professional English Explanation

Fail-fast iterators (used by most standard collections — `ArrayList` , `HashMap` , `HashSet`) detect structural modification during iteration (any add/remove other than via the iterator's own `remove()`) via an internal `modCount` field, and immediately throw `ConcurrentModificationException` — a deliberate design choice to surface bugs early rather than allow undefined behavior silently. **Fail-safe** iterators (`CopyOnWriteArrayList` , `ConcurrentHashMap` , Book 08) iterate over a snapshot or otherwise tolerate concurrent modification without throwing, at the cost of iteration possibly not reflecting the very latest state.

Java Code

```
import java.util.*;
import java.util.concurrent.CopyOnWriteArrayList;

public class IteratorFailFastDemo {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(List.of("a", "b", "c", "d"));

        try {
            for (String s : list) {
                if (s.equals("b")) list.remove(s);           // structural modification during
iteration
            }
        } catch (ConcurrentModificationException e) {
            System.out.println("Fail-fast: " + e.getClass().getSimpleName());
        }

        // CORRECT way: use the Iterator's own remove()
        Iterator<String> it = list.iterator();
        while (it.hasNext()) {
            if (it.next().equals("b")) it.remove();           // safe - iterator tracks this
internally
        }
        System.out.println("After safe removal: " + list);

        // Fail-safe example
        List<String> cowList = new CopyOnWriteArrayList<>(List.of("x", "y", "z"));
        for (String s : cowList) {
            if (s.equals("y")) cowList.remove(s);             // NO exception - iterates over
a snapshot
        }
        System.out.println("CopyOnWriteArrayList after modification during iteration: " +
cowList);
    }
}
```

Output

```
Fail-fast: ConcurrentModificationException
After safe removal: [a, c, d]
CopyOnWriteArrayList after modification during iteration: [x, z]
```

Internal Working

- Fail-fast iterators check an internal `modCount` (a counter incremented on every structural modification) at each `next()` call; if it doesn't match the value captured when the iterator was created, `ConcurrentModificationException` is thrown — this is a **best-effort** detection mechanism, not a strict guarantee (the JavaDoc explicitly says so), but reliably catches the common single-threaded "forgot to use `Iterator.remove()`" bug.
- `Iterator.remove()` is safe precisely because it updates the iterator's own expected `modCount` snapshot at the same time it modifies the backing collection, keeping them in sync.
- `CopyOnWriteArrayList` achieves fail-safe behavior by copying the **entire underlying array** on every mutation (`add` / `remove`) — iterators hold a reference to the array snapshot from when

they were created, completely unaffected by subsequent mutations; this makes writes expensive ($O(n)$ copy each time) but reads/iteration extremely cheap and safe, ideal for read-heavy, write-rare scenarios (Book 08).

Real-World Example

Telugu: `for (Item item : list) { if (condition) list.remove(item); }` — ఇది అత్యంత common Java beginner bug, `ConcurrentModificationException` throw చేస్తుంది. Correct fix: `Iterator.remove()` వాడటం, లేదా `removeIf()` (Java 8+) వాడటం. English: The `for (Item item : list) { if (condition) list.remove(item); }` pattern is one of the most common real Java bugs, throwing `ConcurrentModificationException` — the idiomatic fixes are `Iterator.remove()` (shown above) or, more concisely in modern Java, `list.removeIf(condition)` (Book 07).

Interview Answer

"Fail-fast iterators (used by `ArrayList`, `HashMap`, etc.) detect structural modification during iteration via an internal modification counter and throw `ConcurrentModificationException` as a best-effort bug-detection mechanism. Fail-safe iterators (`CopyOnWriteArrayList`, `ConcurrentHashMap`) tolerate concurrent modification — often by iterating a snapshot — without throwing, trading some memory/consistency guarantees for safety under concurrent or in-loop modification."

Cross Questions

- Q: Is `ConcurrentModificationException` guaranteed to be thrown on every structural modification during iteration? → A: No — it's explicitly documented as best-effort; relying on it for correctness (rather than as a bug-detection aid) is itself bad practice.
- Q: Why is `Iterator.remove()` safe but `list.remove(element)` inside a for-each loop isn't? → A: `Iterator.remove()` updates the iterator's tracked mod-count in sync with the removal; a direct collection-level `remove()` call bypasses the iterator entirely, desynchronizing its expected state.
- Q: Does `CopyOnWriteArrayList` guarantee an iterator sees the latest writes made during its iteration? → A: No — by design, it iterates the snapshot taken when the iterator was created; later writes are simply invisible to that already-in-progress iteration (not an error, just a different consistency model).

Tricky Questions

- Q: If you add (not remove) exactly one element and then immediately stop iterating (break out of the loop) right after, does `ConcurrentModificationException` still get thrown? → A: No — the check happens on the *next* call to `next()` / `hasNext()` after modification; if iteration ends (via `break` or natural completion) before that next call, no exception occurs, even though the modification happened.
- Q: Is a single-threaded program safe from `ConcurrentModificationException`? → A: No — despite the name suggesting "concurrent" (multi-threaded), this exception very commonly occurs in purely single-threaded code, simply from directly modifying a collection while iterating it in the same thread.

Coding Exercise

L1: Reproduce the classic `ConcurrentModificationException` bug and fix it with `Iterator.remove()`. **L2:** Rewrite the same fix using `list.removeIf(condition)` (Java 8+, previewed here, detailed in Book 07). **L3:** Compare behavior of `ArrayList` vs `CopyOnWriteArrayList` when modified during iteration. **L4 (Interview):** Explain the internal `modCount` mechanism behind fail-fast iterators. **L5 (Senior):** Given a read-heavy, rarely-written shared list accessed by multiple threads, justify choosing `CopyOnWriteArrayList` over `Collections.synchronizedList()`. **L6 (Mastery):** Explain, from memory, why `ConcurrentModificationException` is called "best-effort" and is not a strict correctness guarantee.

CHAPTER 12 — Collections Utility Class & Immutable Collections

Telugu Explanation

`java.util.Collections` అనేది static utility methods కలిగిన class — `sort()`, `reverse()`, `shuffle()`, `binarySearch()`, `max()` / `min()`, `unmodifiableList()`, `synchronizedList()` వంటివి. Java 9+ లో `List.of()`, `Set.of()`, `Map.of()` factory methods truly **immutable** collections create చేస్తాయి — `Collections.unmodifiableList()` కంటే safer (అది ఒక mutable backing list యొక్క view మాత్రమే).

Professional English Explanation

`java.util.Collections` provides static utility methods: `sort()`, `reverse()`, `shuffle()`, `binarySearch()`, `max()` / `min()`, `unmodifiableXxx()` (read-only *views*), and `synchronizedXxx()` (legacy synchronization wrappers). Since **Java 9**, `List.of(...)`, `Set.of(...)`, and `Map.of(...)` create **genuinely immutable** collections — distinct from and generally safer than `Collections.unmodifiableList()`, which only wraps a *view* over a backing list that can still be mutated through the original reference.

Java Code

```
import java.util.*;

public class CollectionsUtilityDemo {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>(List.of(5, 2, 8, 1, 9));
        Collections.sort(numbers);
        System.out.println("Sorted: " + numbers);
        Collections.reverse(numbers);
        System.out.println("Reversed: " + numbers);
        System.out.println("Max: " + Collections.max(numbers) + ", Min: " +
Collections.min(numbers));
        System.out.println("Binary search for 8 (must be sorted ascending first): "
            + Collections.binarySearch(new ArrayList<>(List.of(1, 2, 5, 8, 9)), 8));

        // unmodifiableList: a VIEW - the backing list can still change!
        List<Integer> backing = new ArrayList<>(List.of(1, 2, 3));
        List<Integer> view = Collections.unmodifiableList(backing);
        backing.add(4); // legal - modifies the
backing list
        System.out.println("Unmodifiable VIEW reflects backing change: " + view);
        try {
            view.add(5); // illegal - throws
        } catch (UnsupportedOperationException e) {
            System.out.println("Direct modification of the view still blocked: " +
e.getClass().getSimpleName());
        }

        // List.of(): TRUE immutability - no backing list can ever change it
        List<Integer> trueImmutable = List.of(1, 2, 3);
        try {
            trueImmutable.add(4);
        } catch (UnsupportedOperationException e) {
            System.out.println("List.of() is genuinely immutable: " +
e.getClass().getSimpleName());
        }
    }
}
```

Output

```
Sorted: [1, 2, 5, 8, 9]
Reversed: [9, 8, 5, 2, 1]
Max: 9, Min: 1
Binary search for 8 (must be sorted ascending first): 3
Unmodifiable VIEW reflects backing change: [1, 2, 3, 4]
Direct modification of the view still blocked: UnsupportedOperationException
```

Internal Working

- `Collections.unmodifiableList()` (and friends) return a thin **wrapper object** that delegates reads to the backing list but throws `UnsupportedOperationException` on any write attempted *through the wrapper itself* — it does **not** protect against changes made directly to the original backing reference, which is a well-known, genuine gotcha.

- `List.of()` / `Set.of()` / `Map.of()` (Java 9+) return dedicated, compact immutable implementation classes with **no** backing mutable structure at all — there is no "original reference" that could ever mutate them, and they additionally reject `null` elements (throwing `NullPointerException` at creation if any argument is `null`), unlike `Arrays.asList()` or mutable collections.
- `Collections.synchronizedList()` / `synchronizedMap()` wrap a collection with synchronized methods (similar coarse-grained locking philosophy to `Vector` / `Hashtable`, Ch.3/7) — legacy-style thread safety, generally superseded by proper concurrent collections (Book 08) for real concurrent workloads.

Real-World Example

Telugu: API response నా internal mutable list ని నేరుగా return చేయకుండా, `List.copyOf()` లేదా `List.of(...)` వాడి truly immutable snapshot return చేయడం — caller code accidentally internal state మార్చకుండా ఆపడానికి ఇది ఒక security/correctness best practice. English: Returning `List.copyOf(internalMutableList)` (or building with `List.of(...)`) instead of the raw internal mutable list from a public API method is a genuine defensive-programming best practice — it prevents external callers from accidentally (or maliciously) mutating your internal state through a returned reference, directly connecting back to Book 02's encapsulation "leaky getter" discussion.

Interview Answer

"`Collections` provides static utilities like `sort()`, `reverse()`, `binarySearch()`, and `unmodifiableXxx()` / `synchronizedXxx()` wrappers. Critically, `Collections.unmodifiableList()` only creates a read-only view — the backing list can still be mutated through its original reference. Since Java 9, `List.of()` / `Set.of()` / `Map.of()` create genuinely immutable collections with no mutable backing structure at all, and they reject `null` elements outright."

Cross Questions

- Q: What's the practical risk of `Collections.unmodifiableList()` versus `List.of()`? → A: If any code still holds a reference to the original backing list, it can mutate it, and that change will be visible (and confusing) through the supposedly "unmodifiable" view — `List.of()` has no such backdoor.
- Q: Does `List.of()` allow `null` elements? → A: No — it throws `NullPointerException` immediately if any argument is `null`, unlike most mutable `List` implementations.
- Q: Is `Collections.synchronizedList()` sufficient for compound thread-safe operations (like check-then-add)? → A: No — like `Vector`, individual method calls are synchronized, but compound operations still need external synchronization (e.g., synchronizing on the list itself for a multi-step operation) or a proper concurrent collection (Book 08).

Tricky Questions

- Q: If you iterate a `Collections.unmodifiableList()` view while the backing list is concurrently modified elsewhere, what happens? → A: You can still get `ConcurrentModificationException` (Ch.11), since the view delegates iteration to the backing list's own iterator, which is still fail-fast

— "unmodifiable" only blocks writes through the view, it says nothing about concurrent modification safety.

- Q: Does `List.copyOf(list)` create a defensive copy even if `list` is already immutable? → A: The JDK implementation is permitted to (and often does) simply return the same instance if it detects the input is already one of its own immutable implementations, as a safe optimization — behaviorally indistinguishable to calling code either way.

Coding Exercise

L1: Use `Collections.sort()`, `reverse()`, `max()`, `min()`, and `binarySearch()` on a list of integers.

L2: Reproduce the "unmodifiable view still changes via backing reference" gotcha and explain why it happens. **L3:** Refactor a method returning a raw internal mutable list to instead return

`List.copyOf(...)`. **L4 (Interview):** Explain the difference between

`Collections.unmodifiableList()` and `List.of()`. **L5 (Senior):** Review a public API method that returns its internal `ArrayList` field directly — explain the encapsulation risk (bridging to Book 02) and fix it using immutable collections.

L6 (Mastery): Explain, from memory, why `List.of()` is "genuinely" immutable while `Collections.unmodifiableList()` is only a view.

CHAPTER 13 — Choosing the Right Collection: Big-O Decision Framework

Telugu Explanation

Real-world code రాసేటప్పుడు, "ఏ collection వాడాలి?" అనేది requirements మీద ఆధారపడి ఉంటుంది: duplicates కావాలా? order కావాలా? sorted order కావాలా? fast lookup కావాలా? index-based access కావాలా? ఈ chapter లో ఒక **decision framework** మరియు complete **Big-O comparison table** ఇస్తాము.

Professional English Explanation

Choosing the right collection is a requirements-driven decision: do you need duplicates? insertion order? sorted order? fast key-based lookup? indexed access? This chapter consolidates the entire book into one decision framework and a complete Big-O comparison table.

Decision Framework

```
Need key-value associations?
  YES -> Need thread-safety? YES -> ConcurrentHashMap (Ch.8)
                                NO  -> Need sorted keys? YES -> TreeMap (Ch.7)
                                                NO  -> Need insertion/access order? YES
-> LinkedHashMap (Ch.7)

NO -> HashMap (Ch.6)
  NO -> Need to allow duplicates?
    YES -> Need indexed access / frequent middle inserts?
      Indexed access more common -> ArrayList (Ch.2)
      Frequent end insert/remove -> LinkedList / ArrayDeque (Ch.3, Ch.9)
      Priority-based processing -> PriorityQueue (Ch.9)
    NO -> Need sorted order? YES -> TreeSet (Ch.5)
                                NO  -> Need insertion order? YES -> LinkedHashSet (Ch.5)
                                                NO  -> HashSet (Ch.4)
```

Full Big-O Comparison Table

Collection	get/access	add	remove	contains	Ordered?	Duplicates?	Null allowed?
ArrayList	O(1)	Amortized O(1) end / O(n) middle	O(n)	O(n)	Insertion order	Yes	Yes
LinkedList	O(n)	O(1) ends	O(1) ends	O(n)	Insertion order	Yes	Yes
HashSet	—	O(1) avg	O(1) avg	O(1) avg	None	No	One null
LinkedHashSet	—	O(1) avg	O(1) avg	O(1) avg	Insertion order	No	One null
TreeSet	—	O(log n)	O(log n)	O(log n)	Sorted	No	No (usually)

Collection	get/access	add	remove	contains	Ordered?	Duplicates?	Null allowed?
HashMap	O(1) avg	O(1) avg	O(1) avg	O(1) avg (key)	None	Keys: No, Values: Yes	One null key
LinkedHashMap	O(1) avg	O(1) avg	O(1) avg	O(1) avg	Insertion/access order	Keys: No	One null key
TreeMap	O(log n)	O(log n)	O(log n)	O(log n)	Sorted by key	Keys: No	No null key
Hashtable	O(1) avg	O(1) avg, sync	O(1) avg, sync	O(1) avg	None	Keys: No	No
ConcurrentHashMap	O(1) avg	O(1) avg, concurrent	O(1) avg	O(1) avg	None	Keys: No	No
ArrayDeque	—	O(1) ends	O(1) ends	O(n)	Insertion order	Yes	No
PriorityQueue	O(1) peek	O(log n)	O(log n)	O(n)	Heap order (not sorted iteration)	Yes	No

Java Code — A Worked Requirements-to-Collection Example

```
import java.util.*;

public class CollectionChoiceDemo {
    public static void main(String[] args) {
        // Requirement 1: "unique usernames, need fast existence checks, order doesn't matter"
        Set<String> usernames = new HashSet<>();

        // Requirement 2: "recently-viewed products, most-recent-first, bounded to 10, unique"
        Set<String> recentlyViewed = new LinkedHashSet<>();

        // Requirement 3: "leaderboard, always need sorted-by-score view"
        Set<String> leaderboard = new TreeSet<>(Comparator.comparingInt(String::length)); // simplified example

        // Requirement 4: "cache by product ID, fast lookup, no ordering needed"
        Map<String, Double> priceCache = new HashMap<>();

        // Requirement 5: "process print jobs by priority"
        Queue<String> printQueue = new PriorityQueue<>();

        System.out.println("All 5 collections chosen based on requirements, not habit.");
    }
}
```

Real-World Example

Telugu: Interview లో "HashMap vs TreeMap ఎప్పుడు వాడతారు?" అడిగితే, సరైన సమాధానం requirement మీద depend అవుతుంది అని చెప్పడం — ఇదే senior-level thinking, ఏదో ఒకటి "always better" అని కాదు.

English: When an interviewer asks "when would you use `HashMap` vs `TreeMap` ?", the senior-level answer is always requirements-driven ("it depends on whether sorted iteration matters enough to pay $O(\log n)$ instead of $O(1)$ ") — never "X is always better than Y," which signals junior-level pattern-matching rather than genuine understanding.

Interview Answer

"Collection choice is requirements-driven: need key-value lookup vs a flat group, duplicates allowed or not, sorted vs insertion vs no ordering guarantee, thread-safety needs, and the actual access pattern (indexed vs end-based vs priority-based) all point to a specific collection via well-understood Big-O trade-offs — there's no universally 'best' collection."

Cross Questions

- Q: When would you choose `TreeMap` over `HashMap` despite the $O(\log n)$ cost? → A: When you genuinely need sorted-by-key iteration or range queries (`firstKey` , `ceilingKey` , etc.) as a core requirement, not just as a nice-to-have.
- Q: When would `LinkedList` outperform `ArrayList` in practice? → A: Workloads dominated by insertions/removals at the ends (queues, dequeues, undo stacks) rather than indexed access or `contains()` .
- Q: Why might you still choose `ArrayList` even for a workload with frequent middle insertions, contrary to the "textbook" advice? → A: For small lists, `ArrayList` 's better cache locality often outperforms `LinkedList` in real benchmarks despite the "worse" Big-O, since $O(n)$ array shifting on a small, cache-resident array can beat $O(1)$ pointer-chasing through scattered heap nodes — Big-O isn't the whole performance story at small scale.

Tricky Questions

- Q: Is `HashMap` always the fastest choice for key-value storage? → A: Not necessarily — for a very small, fixed set of keys, a simple array-backed or even `switch` -based lookup can outperform `HashMap` 's hashing overhead; for very large, well-distributed datasets, `HashMap` wins decisively. "It depends on scale" is a genuinely correct senior answer.
- Q: If sorted order is only needed occasionally (e.g., for one report), should you use `TreeMap` for the whole application? → A: Not necessarily — you could use `HashMap` for the frequent $O(1)$ operations and sort a snapshot (`new ArrayList<>(map.entrySet())` + `Collections.sort()`) only when the occasional sorted view is actually needed, avoiding paying $O(\log n)$ on every operation.

Coding Exercise

L1: For 5 given requirements (provided or self-written), pick the correct collection and justify using this chapter's decision framework. **L2:** Benchmark `ArrayList` vs `LinkedList` for 100,000 middle-insertions and reconcile the (possibly surprising) result with Big-O theory. **L3:** Refactor a method using `TreeMap` for all operations when only one report needs sorted order — switch to `HashMap` +

on-demand sorting. **L4 (Interview):** Recreate the full Big-O comparison table from memory for at least 6 collections. **L5 (Senior):** Given a real system design scenario (e.g., a leaderboard service with 10M users, updated 1000x/sec, read rarely), design the collection/data-structure strategy and justify every choice. **L6 (Mastery):** Explain, from memory, why "it depends on the requirement" is the only universally correct answer to "which collection is best," and demonstrate you can actually apply the dependency, not just recite the phrase.

CHAPTER 14 — Mini Project: In-Memory Inventory System

Goal

Combine every collection type from this book into one cohesive, realistic system — no database (Book 09 covers that), pure in-memory Collections Framework usage.

Requirements

1. `Product` class (id, name, category, price, stock) with proper `equals()` / `hashCode()` by `id` (Book 01/02) and `Comparable` by `price` (Ch.10).
2. `InventoryService` internally using:
 - `Map<String, Product> productsById` (`HashMap`, Ch.6) — fast lookup by ID.
 - `Map<String, Set<Product>> productsByCategory` (`HashMap<String, TreeSet<Product>>`, Ch.5–6) — products grouped by category, each group sorted by price.
 - `Set<String> lowStockAlerts` (`LinkedHashSet`, Ch.5) — insertion-ordered list of product IDs currently below a stock threshold.
 - A bounded `LinkedHashMap`-based LRU cache (Ch.7) of the 5 most-recently-viewed products.
3. Methods: `addProduct()`, `getProduct(id)` (updates the LRU cache), `updateStock()` (updates `lowStockAlerts` as needed), `getProductsByCategorySorted(category)`, `getTopNCheapest(n)` (using a `PriorityQueue`, Ch.9).
4. Use `Comparator` chaining (Ch.10) to implement a `getProductsSorted(Comparator<Product>)` method supporting multiple sort views without modifying `Product`.
5. Ensure zero `ConcurrentModificationException` bugs (Ch.11) — use `Iterator.remove()` or `removeIf()` wherever removal-during-iteration is needed.
6. Return defensive immutable copies (`List.copyOf()`, Ch.12) from any method exposing internal collections.

Concepts Reinforced

Every chapter in this book — hierarchy navigation (Ch.1), `ArrayList` / `LinkedList` trade-offs (Ch.2–3), `HashSet` / `TreeSet` (Ch.4–5), `HashMap` internals (Ch.6), `LinkedHashMap` LRU (Ch.7), `PriorityQueue` (Ch.9), `Comparable` / `Comparator` (Ch.10), safe iteration (Ch.11), immutability (Ch.12), and the Big-O decision framework (Ch.13) driving every structural choice above.

Stretch Goals

- Add thread-safety using `ConcurrentHashMap` (Ch.8) and note which parts of the design would need to change (full correctness detail arrives in Book 08).
- Add a `searchByNamePrefix()` method using `TreeMap`'s `subMap()` for efficient prefix-range queries.

FINAL REVISION NOTES

- **Map is not a Collection** — separate hierarchy, `keySet()` / `values()` / `entrySet()` bridge the two.
 - **ArrayList** : $O(1)$ get, amortized $O(1)$ end-add, $O(n)$ middle insert/remove. **LinkedList** : opposite trade-off, $O(1)$ at ends, $O(n)$ indexed access.
 - **HashSet** is literally a **HashMap** in disguise — same $O(1)$ average, same equals/hashCode dependency.
 - **LinkedHashSet / LinkedHashMap** : insertion order (or access order for LHM) at the same average cost as their unordered counterparts.
 - **TreeSet / TreeMap** : sorted via Red-Black Tree, $O(\log n)$, require **Comparable** / **Comparator** ; uniqueness governed by `compareTo()` , not `equals()` .
 - **HashMap internals**: hash-spread $\rightarrow$ bucket index via `(capacity-1)&hash` $\rightarrow$ chain or (Java 8+) tree on 8+ collisions in a 64+ bucket table $\rightarrow$ resize (double) at 0.75 load factor.
 - **ConcurrentHashMap** : modern fine-grained locking + CAS, no `null` keys/values, atomic compound ops (`merge` , `computeIfAbsent`) — full mechanics in Book 08.
 - **ArrayDeque** > **Stack** / **LinkedList** for stack/queue usage (better cache locality, JDK-recommended). **PriorityQueue** : binary heap, $O(\log n)$, iteration $\neq$ sorted order.
 - **Comparable** : one natural order, inside the class. **Comparator** : unlimited external orderings.
 - **Fail-fast** (**ArrayList** , **HashMap** ...) throws **ConcurrentModificationException** on in-loop structural modification (best-effort); **fail-safe** (**CopyOnWriteArrayList** , **ConcurrentHashMap**) tolerates it via snapshots/fine-grained safety.
 - **Collections.unmodifiableList()** is only a view (backing list can still mutate); **List.of()** (Java 9+) is genuinely immutable and null-hostile.
 - **Choosing a collection is always requirements-driven** — no universal "best," only correct trade-offs for the actual access pattern.
-



CHEAT SHEET

```
Collection hierarchy: Iterable -> Collection -> List/Set/Queue | Map is SEPARATE
ArrayList: O(1) get, amortized O(1) end-add, O(n) mid insert/remove
LinkedList: O(1) at ends, O(n) indexed access; implements Deque too
HashSet = HashMap in disguise (keys only) - O(1) avg, depends on good hashCode>equals
LinkedHashSet/Map: insertion(or access, for Map) order, same O(1) avg cost
TreeSet/Map: Red-Black Tree, O(log n), sorted, uniqueness via compareTo() not equals()
HashMap: hash-spread -> (cap-1)&hash bucket -> chain/tree(8+, cap>=64) -> resize at 0.75 load factor
ConcurrentHashMap: fine-grained lock+CAS, no null, use merge()/computeIfAbsent() for atomic compound ops
ArrayDeque: modern Stack/Queue replacement, circular array, no null
PriorityQueue: binary heap, O(log n), poll()=priority order, iteration != sorted
Comparable: ONE natural order (in-class) | Comparator: MANY orders (external, chainable)
Fail-fast (ArrayList/HashMap): ConcurrentModificationException on bad in-loop mutation (best-effort)
Fail-safe (CopyOnWriteArrayList/ConcurrentHashMap): tolerates concurrent/in-loop mutation
Collections.unmodifiableXxx() = VIEW (backing can still change) | List.of()/Set.of()/Map.of() = TRUE immutable, null-hostile
```

INTERVIEW QUESTION BANK — Collections Framework

Beginner

1. What is the difference between `List`, `Set`, and `Map`?
2. Why is `ArrayList` faster for `get()` but slower for middle insertion than `LinkedList`?
3. What is the default load factor of a `HashMap`, and what does it control?
4. What is the difference between `Comparable` and `Comparator`?
5. What does `ConcurrentModificationException` mean, and when does it occur?

Intermediate 6. Explain how `HashSet` is internally implemented using `HashMap`. 7. Explain the full `HashMap.put()` algorithm, including hash spreading and treeification. 8. How does `LinkedHashMap` implement LRU cache behavior? 9. What is the difference between `Collections.unmodifiableList()` and `List.of()`? 10. Why does `TreeSet` / `TreeMap` use `compareTo()` instead of `equals()` for uniqueness?

Advanced 11. Explain treeification in `HashMap` — what triggers it, and what problem does it solve? 12. Why is `ConcurrentHashMap` null-hostile while `HashMap` allows one null key? 13. Explain why `PriorityQueue`'s iteration order is not the same as its `poll()` order. 14. Compare the locking strategy evolution of `ConcurrentHashMap` from pre-Java-8 to Java-8+. 15. Why is `ArrayDeque` generally preferred over both `Stack` and `LinkedList` for stack/queue use?

Senior/Architect 16. Design the collection/data-structure strategy for a real-time leaderboard serving 10M users with 1000 writes/sec. 17. A production `HashMap`-backed cache is showing $O(n)$ lookup degradation under load — diagnose the likely root cause and fix it. 18. Explain the full Big-O decision framework for choosing between `HashMap`, `TreeMap`, and `LinkedHashMap` for a new feature, given specific requirements you're handed in the interview. 19. Design a thread-safe, bounded LRU cache from scratch, explaining what changes from the single-threaded `LinkedHashMap` version and why (bridging to Book 08). 20. Review a codebase returning internal mutable collections directly from public API methods — propose and justify a fix using this book's immutability tools.

(Full short/professional/deep-senior answer scaffolding for every question across the series lives in Book 23 — Java Interview Master Book.)



CROSS-QUESTION ENGINE — Sample Chains

Q: What is a HashMap? → Q: How does it work internally? → Q: What happens when two keys have the same hash? → Q: Why are `equals()` and `hashCode()` related? → Q: What happens when the HashMap grows? → Q: What is load factor? → Q: Why is HashMap not thread-safe? → Q: What would you use in a concurrent application?

Q: What is the difference between ArrayList and LinkedList? → Q: Which has better `get()` performance and why? → Q: Which has better insertion at the ends and why? → Q: When would you actually choose LinkedList in practice? → Q: Is that choice always faster in real benchmarks?

Q: What is the difference between fail-fast and fail-safe iterators? → Q: Give an example of each. → Q: What causes `ConcurrentModificationException`? → Q: How do you safely remove elements while iterating? → Q: Why is `CopyOnWriteArrayList`'s write cost higher?



CONSOLIDATED EXERCISES (All Levels)

L1 — Beginner: Redo each chapter's L1 exercise unaided. **L2 — Intermediate:** Redo each chapter's L2/L3 exercises, explaining every internal mechanic out loud in Telugu. **L3 — Advanced:** Implement a from-scratch simplified `HashMap` (array of buckets + linked-list collision chains, no treeification needed) to prove you understand Ch.6's internals. **L4 — Interview:** Answer all 20 Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Chapter 14 mini project fully, including both stretch goals. **L6 — Mastery:** Teach Chapters 6 (`HashMap` internals), 10 (`Comparable/Comparator`), and 13 (decision framework) out loud, from memory, to someone else.



ONE-DAY REVISION PLAN (≈6 hours)

Time	Focus
0:00–0:20	Ch.1: Full hierarchy — redraw from memory
0:20–1:00	Ch.2–3: ArrayList/LinkedList/Vector/Stack — memorize the Big-O table
1:00–1:40	Ch.4–5: HashSet/LinkedHashSet/TreeSet
1:40–2:00	Break
2:00–3:00	Ch.6: HashMap internals — the highest-density interview block, re-read twice
3:00–3:40	Ch.7–8: LinkedHashMap/TreeMap/Hashtable/ConcurrentHashMap
3:40–4:10	Ch.9: Queue/Deque/PriorityQueue
4:10–4:40	Ch.10: Comparable vs Comparator — code all 3 comparator variants from the demo
4:40–5:10	Ch.11–12: Fail-fast/fail-safe, Collections utility & immutability
5:10–5:40	Ch.13: Decision framework — recreate the full Big-O table from memory
5:40–6:00	Answer the full Interview Question Bank from memory



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1–3 (hierarchy, ArrayList, LinkedList/Vector/Stack) — benchmark ArrayList vs LinkedList yourself
2	Ch.4–5 (HashSet, LinkedHashSet, TreeSet) — reproduce every equals/hashCode gotcha
3	Ch.6 (HashMap internals) — implement a simplified from-scratch HashMap
4	Ch.7–8 (LinkedHashMap/TreeMap/Hashtable, ConcurrentHashMap) — build the LRU cache from scratch
5	Ch.9–10 (Queue/Deque/PriorityQueue, Comparable/Comparator) — implement top-K and multi-level sorting
6	Ch.11–13 (iterators, immutability, decision framework) + Mini Project — build the full Inventory System
7	Full mock interview: all 20 bank questions + all cross-question chains, in Telugu and English, without notes

FINAL MASTERY CHECKLIST

- ☐ I can draw the full Collections Framework hierarchy from memory, including why `Map` is separate.
- ☐ I can explain the Big-O trade-offs between `ArrayList` and `LinkedList` and justify a real choice.
- ☐ I can explain that `HashSet` is backed by `HashMap` and why `equals()` / `hashCode()` matter for it.
- ☐ I can explain `HashMap` 's full internal algorithm: hashing, bucket selection, collisions, treeification, resizing.
- ☐ I can implement an LRU cache using `LinkedHashMap` 's access-order mode.
- ☐ I can explain `ConcurrentHashMap` 's structural difference from `Hashtable` and `HashMap` .
- ☐ I can use `PriorityQueue` correctly and explain why its iteration order isn't sorted.
- ☐ I can distinguish `Comparable` from `Comparator` and chain multiple comparators.
- ☐ I can explain fail-fast vs fail-safe iterators and safely remove elements during iteration.
- ☐ I can explain the difference between `Collections.unmodifiableList()` and `List.of()` .
- ☐ I can apply the Big-O decision framework to choose the right collection for a new requirement.
- ☐ I built the In-Memory Inventory System mini project, including both stretch goals.
- ☐ I answered the full Interview Question Bank confidently in both Telugu and English.

Once every box is checked, you are ready for `06_Java_Generics.md` .